Complete Start-to-Finish Project Guide

NADRA LinkOps AI

This document explains **everything from start to end** so that you can understand, rebuild, improve, deploy, and submit the project yourself.

---

1. Project Title

NADRA LinkOps AI

2. Project Idea

A web app for a NADRA network engineer in Quetta Region to:

- record daily connectivity complaints,
- identify whether an issue is genuine connectivity or not,
- track monthly disruption reports,
- draft telecom escalation emails using AI,
- track AD OU transfers, unlocks, and password resets.

---

3. Why this is a good final project

It is:

- original,
- based on a real operational problem,
- complete end-to-end,
- useful in Pakistan context,
- strong for viva/presentation,
- and contains a custom AI feature.

---

4. Tools used in this project

Frontend

- HTML5
- CSS3
- Vanilla JavaScript

Storage

- Browser localStorage

AI Integration

- OpenRouter-compatible API
- Serverless function in `api/ai-draft.js`

Version Control

- Git
- GitHub

Deployment

- Vercel

Documentation

- Markdown
- PDF guide
- PNG screenshots

---

5. Project folder structure

```
nadra-linkops-ai/
├── api/
│   └── ai-draft.js
├── assets/
│   └── screenshots/
├── docs/
│   ├── COMPLETE_PROJECT_GUIDE.md
│   ├── COMPLETE_PROJECT_GUIDE.pdf
│   ├── NADRA_LinkOps_AI_Visual_Tutorial.pdf
│   ├── TUTORIAL.md
│   ├── SOURCES.md
│   ├── SUBMISSION_CHECKLIST.md
│   └── system-prompt.md
├── index.html
├── styles.css
├── script.js
├── README.md
└── vercel.json
```

---

6. How to start from zero

If you wanted to build this project yourself from scratch, the correct order is:

1. Understand the problem.
2. Write the feature list.
3. Decide the technology stack.
4. Design the UI layout.
5. Build the frontend screens.
6. Add local data handling.
7. Add AI feature.
8. Write README.
9. Take screenshots.
10. Push to GitHub.
11. Deploy on Vercel.
12. Test and submit.

---

7. Step 1 — Understand the real problem

The user story is:

- DAU incharges call daily and report connectivity issues.
- The engineer checks NMS.
- The engineer identifies whether primary or backup link is affected.
- The engineer contacts PTCL, Wateen, VSAT, DRS, or backup SIM provider.
- The engineer also sends escalation email and informs HQ.
- At month end, management needs a disruption report.
- The same engineer also handles AD tasks like OU transfer and password reset.

This is the business case of the app.

---

8. Step 2 — Convert problem into app modules

The project is divided into these modules:

- Login page
- Dashboard
- Complaint intake page
- Complaint register
- Monthly reports
- AD support tracker
- AI assistant
- Backup/import/export
- Documentation

---

9. Step 3 — Choose the stack

Why this project uses simple web technologies:

- HTML/CSS/JS are easy to understand.
- No heavy framework is required.
- It can deploy easily on Vercel.
- The app is easier to explain in class.
- There is less risk of build errors compared to React/Next for a beginner.

---

10. Step 4 — Design the UI

The UI was designed to look more formal and government-style:

- clean header,
- green NADRA-inspired color palette,
- login screen,
- professional cards,
- structured complaint form,
- readable tables,
- good contrast between labels and input fields.

When designing yourself, always do this:

1. Sketch rough layout on paper.
2. Decide main sections.
3. Keep data-entry forms simple.
4. Hide optional fields in advanced section.
5. Ensure text is readable on all backgrounds.

---

11. Step 5 — Build the HTML structure

Main file: `index.html`

HTML responsibilities:

- login page layout,
- header and navigation,
- dashboard section,
- complaints section,
- AD support section,

- reports section,
- AI section,
- documentation section.

Best practice:

- use clear IDs for all inputs,
- group fields into sections,
- keep labels directly above inputs,
- use semantic tags like `header`, `main`, `section`, `article`, `form`.

---

12. Step 6 — Build the CSS styling

Main file: `styles.css`

CSS responsibilities:

- colors,
- spacing,
- typography,
- cards,
- buttons,
- form appearance,
- tables,
- responsive design.

Good UI rules:

- dark text on white input fields,
- enough padding inside fields,
- distinguish required vs optional fields,
- consistent button styles,
- mobile responsive layout.

---

13. Step 7 — Build JavaScript logic

Main file: `script.js`

JavaScript responsibilities:

- login handling,
- tab switching,
- complaint save/edit/delete,

- AD request save/edit/delete,
- localStorage save/load,
- dashboard calculations,
- monthly report generation,
- export CSV,
- export/import JSON,
- AI request handling.

Important concept:

The app is data-driven. That means dashboard and reports are calculated from saved complaint records.

---

14. Step 8 — Data model design

Complaint fields

- id
- dauName
- district
- inchargeName
- inchargeContact
- complaintId
- telecomTicket
- linkRole
- provider
- issueNature
- validatedCause
- disruptionStart
- resolutionTime
- status
- userImpact
- remarks

AD request fields

- id
- employeeName
- employeeRef
- oldDau
- newDau

- requestType
- hrOrder
- requestedBy
- status
- requestDate
- completedDate
- notes

Always define your data structure before coding.

---

15. Step 9 — Add sample data

Why sample data is important:

- the app looks complete immediately,
- screenshots become easy,
- grader can test without entering everything manually,
- dashboard and reports show meaningful output.

This project includes realistic Pakistan examples for Quetta Region.

---

16. Step 10 — Add public site directory

This project includes a Quetta-region site directory based on publicly searchable NADRA office listings.

Important note:

- Public site names and addresses were found.
- Public incharge names were not reliably found.
- Therefore the app lets the user enter or replace incharge names manually.

This is a good academic decision because it avoids fake official data.

---

17. Step 11 — Add login screen

A demo login page was added because it makes the project feel complete.

Demo credentials:

- `engineer@nadra.pk / Pass@123`
- `admin@nadra.pk / Admin@123`

Important:

This is demo authentication stored in localStorage, not a secure production login system.

That is acceptable for an academic project unless real authentication is required.

---

18. Step 12 — Add the AI feature

Files involved:

- `script.js`
- `api/ai-draft.js`
- `docs/system-prompt.md`

What AI does:

- classifies complaint,
- estimates severity,
- drafts escalation email,
- drafts DAU update,
- suggests next steps.

Why it satisfies the assignment:

The prompt was written specifically for this use case instead of using a generic AI widget.

---

19. Step 13 — Write the system prompt

Your AI feature is stronger when the prompt tells the model exactly what to do.

A good prompt should include:

- role of the AI,
- type of input,
- exact tasks,
- required output format,
- tone and rules.

This project uses a structured system prompt so the output remains professional and useful.

---

20. Step 14 — Build the API route

File: `api/ai-draft.js`

Why an API route is needed:

- API keys should not be exposed in frontend JavaScript.
- Vercel serverless functions can securely read environment variables.

The flow is:

1. frontend sends complaint data,
2. serverless function reads data,
3. function calls AI provider,
4. response returns to browser,
5. UI shows the AI output.

---

21. Step 15 — Add fallback logic

The app includes fallback AI text if no live API key is available.

Why this is useful:

- app still works during demo,
- teacher can still see the feature,
- development is easier,
- fewer runtime failures before deployment.

---

22. Step 16 — Test locally

Option A

Open `index.html` directly.

Option B

Use a local web server:

python -m http.server 8000

Then open:

http://localhost:8000

What to test:

- login works,
- complaint form saves,
- complaint register updates,
- AI draft works,
- report generation works,
- exports work,
- AD support tracker works.

---

23. Step 17 — Create screenshots

The README requires at least 3 screenshots.

Take screenshots of:

- dashboard,
- complaint form/register,
- AI assistant,
- reports page.

Make sure screenshots match the final deployed version.

---

24. Step 18 — Write the README properly

README must include:

- app name,
- problem statement,
- live URL,
- features,
- AI feature,
- tools/services/models,
- screenshots,
- run instructions.

Many students lose marks because README is weak even if the app is good.

---

25. Step 19 — Push to GitHub

Commands:

```
git init
git add .
git commit -m "Initial commit - NADRA LinkOps AI"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/nadra-linkops-ai.git
git push -u origin main
```

Rules:

- repository must be public,
- never commit API keys,

- check repo in incognito mode.

---

26. Step 20 — Deploy to Vercel

Deployment process:

1. Sign in to Vercel.
2. Import GitHub repository.
3. Deploy project.
4. Add environment variables.
5. Redeploy.

Required environment variables:

- `OPENROUTER_API_KEY`
- `OPENROUTER_MODEL`
- `PUBLIC_APP_URL`

Example model:

- `openai/gpt-4.1-mini`

---

27. Step 21 — Final update before submission

Update `README.md` with:

- real GitHub repo URL,
- real live Vercel URL,
- final screenshots if needed.

Then test:

- GitHub repo in incognito,
- live deployed app in incognito.

---

28. Step 22 — Submission process

The assignment says submit only your public GitHub repository link.

So the final portal submission should be:

https://github.com/your-username/nadra-linkops-ai

Do not submit:

- a zip file,

- a private repo,
- a broken Vercel URL,
- someone else’s project.

---

29. Common mistakes to avoid

- private repository,
- missing live URL,
- broken README,
- fake AI feature,
- no screenshots,
- committing API keys,
- incomplete app,
- placeholder links left unchanged.

---

30. If you want to improve it further

Possible future improvements:

- real database (Supabase/PostgreSQL),
- role-based authentication,
- PDF export from backend,
- telecom SLA tracking,
- email sending integration,
- provider-wise trend charts,
- real user management,
- audit logs.

---

31. Best viva explanation

You can explain your project like this:

> My project is NADRA LinkOps AI. It solves a real network operations problem in Pakistan. A NADRA regional network engineer receives daily complaints from DAUs about PTCL, Wateen, VSAT, DRS, and 3G/4G backup links. My app records complaints, tracks disruption reports, identifies genuine vs non-connectivity issues, generates monthly reports, tracks AD support tasks, and uses AI to draft telecom escalation emails and DAU updates.

---

32. Final checklist

Before submission, confirm:

- project works locally,
- repo is public,
- live URL works,
- README is complete,
- screenshots are present,
- API key is not committed,
- GitHub link opens without login.

---

33. Final conclusion

This project is strong because it combines:

- originality,
- real Pakistan context,
- technical completion,
- deployment readiness,
- AI integration,
- and proper documentation.

It is suitable for final submission after you replace placeholder links with your real GitHub and Vercel URLs.